

How SD-Assistant works, from start to end

A friendly computer helper for a big company. People tell it their computer problem in plain words. It fixes the easy ones straight away, and passes the hard ones to the right IT person with everything already written down. This page explains every part of it, simply enough for a ten-year-old.

Contents

- | | | | |
|----------|---------------------|-----------|---------------------------|
| 1 | Why we built it | 9 | Time promises |
| 2 | Who uses it | 10 | The screens |
| 3 | The parts | 11 | Map of the code |
| 4 | A ticket's journey | 12 | Testing and delivery |
| 5 | How the bot decides | 13 | How to run it |
| 6 | Answers and fixes | 14 | What we built, day by day |
| 7 | Safety rules | 15 | Word list |
| 8 | Tricky situations | | |

1 Why we built it

The problem, and the idea that solves it.

Imagine a school with five thousand students and only fifteen teachers who know how to fix computers. Every day, students come with problems: *"I forgot my password."* *"The Wi-Fi isn't working."* *"The printer is stuck."* The teachers spend most of the day answering the same easy questions, and the students wait in a long line.

A big company has the same problem. Its computer helpers are called the **IT Service Desk**. When someone has a problem, IT writes it down as a **ticket**, a numbered note like `TKT-3001`, so nothing gets forgotten.

The idea: put a helpful robot at the front of the line. The robot listens, asks a couple of questions, and walks people through the fix for easy problems. If the fix doesn't work, or the problem needs a real person, it hands the ticket to the right IT team, and that team can already see everything the person said and tried. Nobody has to explain twice.

We call the whole thing **SD-Assistant** ("SD" stands for Service Desk). Its main goals are:

- **Answer first, then a person.** Every problem people can safely fix themselves gets real steps before anyone is called.
- **Never make things up.** Every answer comes from an approved help article or a fixed message, never from the robot's imagination.
- **Never miss something dangerous.** Hacking, a swelling battery or a lost phone go straight to a person, every time.
- **Be honest about time.** Tell people exactly when someone will reply, with the date.

2 Who uses it

Three kinds of people, each with their own screens.

Employee

The student who needs help

Anyone in the company. Chats with the robot, fills in a ticket form, and sees their tickets under **My tickets**.

Agent

The computer teacher

An IT person in one team, for example the VPN team. Sees their team's line of tickets (the **queue**), gets alerts, chats with the employee and fixes the problem.

Supervisor

The head teacher

Sees every team's tickets, charts showing how well things are going, approves new help articles, and manages user accounts.

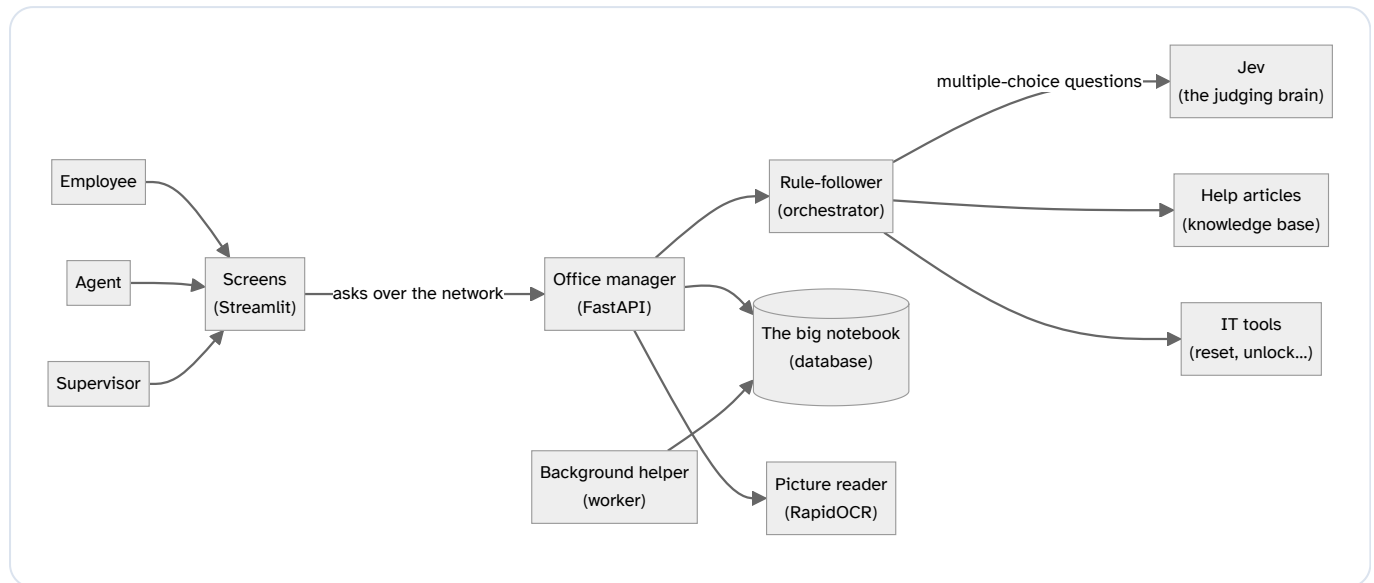
There are 12 kinds of problems (called **categories**), and each one belongs to a team:

Category	Example	Team
Password & Account Access	"I forgot my password", "my account is locked"	Identity Support
VPN & Remote Access	"VPN won't connect from home"	Network Remote Access
Software License & Installation	"I need Zoom installed"	Software Asset Management
Hardware & Peripherals	"my laptop won't turn on"	End User Hardware
MFA & Authentication	"I got a new phone, move my authenticator"	Identity Security
Email & Outlook	"emails stuck in the Outbox"	Messaging Support
Application Access & Permissions	"I need access to Jira"	Application Access
Network & Connectivity	"office Wi-Fi has no internet"	Network Operations
Security Incidents	"I clicked a link in a strange email"	Security Operations
Printer & Scanning	"my print job is stuck"	Workplace/Print Support
Collaboration Tools	"I can't join a Teams meeting"	Collaboration Support
Mobile Device & MDM	"I lost my work phone"	Mobile Device Management

Anything that doesn't fit goes to the **Service Desk Duty Manager**, the "catch-all" person.

3 The parts of the system

Like the rooms of a school, each part has one job.



Everyone uses the screens. The screens only talk to the office manager, and the office manager is the only one allowed to write in the notebook.

The screens

Like the front desk

The web pages people see: the chat, the ticket form, the queue, charts. Built with **Streamlit**, a tool for making web pages in Python.

app.py, views/, ui/

The office manager (the API)

Like the school office

Every request goes through it: "sign me in", "send this chat message", "show my tickets". It checks who you are and what you're allowed to do. Built with **FastAPI**.

api.py

The big notebook (database)

Like the school register

Stores every ticket, message, user and event. Nothing is ever erased; changes are added as new lines, so you can always see what happened. SQLite on a laptop, Postgres on a real server.

servicedesk/store.py

Jev, the judging brain

Like a quiz expert

An AI model from TypeSafe that only answers *multiple-choice* questions, and always says how sure it is ("VPN: 92%"). It never writes sentences, so it can't invent an answer.

servicedesk/brain.py

The rule-follower (orchestrator)

Like a careful receptionist with a rulebook

The heart of the chat. It asks Jev questions, follows the rules, picks the help article, asks for missing details, gives steps, and decides when to call a person.

`servicedesk/orchestrator.py`

Help articles (knowledge base)

Like a recipe book of fixes

31 articles, one per kind of problem. Each has a version written for employees, with real steps, which a supervisor approves before the robot may use it.

`content/kb_employee_drafts.json, services/kb.py`

The rulebooks (datasets)

Like the school's written rules

28 files from the project's specification: the categories, how to work out priority, which team handles what, identity-check levels, and more.

`datasets/`

Picture reader (OCR)

Like reading a photo of the blackboard

When someone attaches a screenshot, it reads the text in it (for example the error message), and blurs anything that looks like a password. It runs on our own computer, so pictures never leave the company.

`services/attachments.py`

IT tools

Like the buttons in the IT office

Reset a password, unlock an account, clear an authenticator. In this demo they are pretend (simulated); in a real company they would be connected to the real systems.

`servicedesk/tools.py`

Background helper (worker)

Like the school bell

Wakes up every 15 seconds in the background: repeats alerts nobody answered, sends emails and Teams messages, confirms and closes old tickets, and once a day deletes old screenshots and personal text on time.

`worker.py`

Watchdog

Like a hall monitor

Keeps checking that the service desk itself is running, and tells IT if it isn't.

healthwatch.py

Admin tool

Like the office's key cabinet

A command-line tool to check the setup, create users, import users, reset passwords and change roles.

manage.py

4 A ticket's journey, start to end

Follow one real problem through the whole system.

Meet Priya. She is working from home and her VPN (the secure tunnel into the company network) won't connect. Here is what happens, step by step.

Priya

My VPN won't connect, it times out

SD-Assistant

To point you to the right fix, I need a couple of details first.

What happens when the VPN fails to connect?

It times out or keeps trying

It shows an error code

Something else, I'll type it

It times out or keeps trying

Thanks. **Which operating system is your laptop on?**

Windows 11

Windows 10

Other

Windows 11

Most of the time the VPN app is holding on to an old connection, and a clean restart fixes it.

1. Close the VPN app completely...
2. Clear its cache...
3. Connect again and approve the sign-in.

Did that fix it?

Yes, it's fixed

No, still not working

Talk to a human

- 1 Priya signs in.** The office manager checks her employee ID and password, and gives her screen a pass (a *token*) that lasts 8 hours, or 30 minutes if she's idle. Five wrong passwords in a row lock the account for 15 minutes, for safety.
- 2 She types her problem in her own words.** She could also attach a screenshot; the picture reader would pull out the error message.
- 3 Safety check first.** Before anything else, Jev checks every message for danger: a password typed into the chat, a hacking attempt, a safety risk like a burning battery, or someone trying to trick the robot. A password gets hidden from the ticket straight away.
- 4 Sorting (triage).** Jev answers multiple-choice questions: Is this a problem, a how-to question, a hello, or not about IT at all? Which of the 12 categories is it? How many people does it affect, and how urgent is it? For Priya: a problem, VPN, just her.
- 5 Priority.** The rule-follower uses a table (the **priority matrix**) to turn "how many people" and "how urgent" into a priority from **P1** (emergency) to **P4** (can wait). Priya's is **P3**.
- 6 A ticket is opened.** It gets a number, like **TKT-3004**, and is written in the big notebook with the right team: Network Remote Access.
- 7 Pick the right help article.** Jev scores how well each VPN article fits ("VPN won't connect: 91%"). The best one is chosen, as long as it scores high enough.
- 8 Ask only what's missing, with buttons.** The article needs a couple of details. Jev checks which ones Priya already gave; the robot asks only for the rest. Things it already knows, like her employee ID or laptop tag, come from her profile.
- 9 Attempt 1.** The robot shows the first fix, written for employees, and asks "Did that fix it?"
- 10 Attempt 2, if needed.** If it didn't work, the robot tries a different fix from the same article.
- 11 Fixed?** If yes, the ticket is marked fixed (Priya can reopen it within 7 days). If she says "*thank you*", the chat closes, and her next message starts a fresh conversation.
- 12 Not fixed? A hand-off.** The robot passes the ticket to the team and tells Priya who has it, the priority, and exactly when to expect a reply ("by 10:00 am on Monday, 5 Oct"). It packs a **hand-off note** with everything: her words, her answers, both attempts and why it handed over.

13

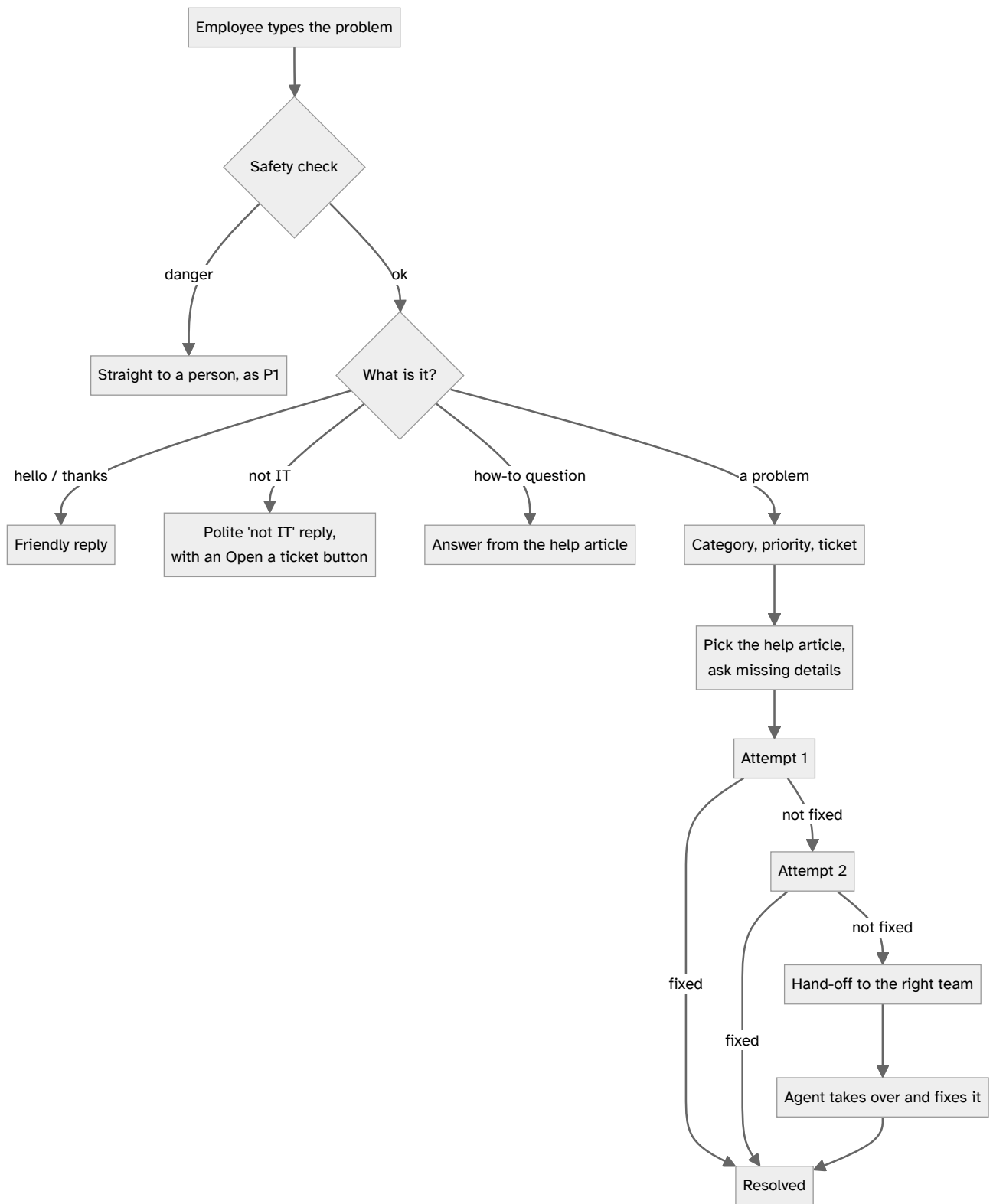
The team is alerted. How loudly depends on priority: a **P1** pops up a dialog for every agent in the team and repeats every 2 minutes until someone answers; a **P2** shows a pop-up note; **P3** and **P4** show a counter.

14

An agent takes over. Ananya, a VPN agent, presses **Take over**. She reads the whole chat and the robot's reasoning, replies to Priya (her message appears in Priya's chat), and ticks a checklist of things to do.

15

Resolved and confirmed. Ananya can only press **Resolve** once the checklist and notes are done. Priya is asked to confirm it's fixed and to rate the help from 1 to 5.



The whole journey in one picture. Some problems skip the attempts on purpose (see Safety rules).

5 How the bot decides

A brain that only answers quizzes, and a rule-follower that makes the decisions.

Most chatbots write their own sentences, so sometimes they invent things that aren't true. SD-Assistant splits the job in two:

Jev judges

Jev only answers questions like "*Which category is this? Options: VPN, Email, Printer...*" and gives a percentage for each option. It's like a very good quiz player who never writes essays.

The code decides

The rule-follower reads those percentages and applies fixed rules: if Jev is less than 50% sure of the category, ask the person instead of guessing; if no article fits well, ask one clarifying question, then call a person.

Some of the questions the code asks Jev:

Question to Jev	What the code does with the answer
Is this message dangerous? (password, hacking, safety, trickery)	Hide passwords; send danger straight to a person
What does the person want? (problem, question, hello, not IT)	Choose the right kind of reply
Which category? How many affected? How urgent?	Choose the team and work out the priority
How well does each help article fit?	Pick the best one, or ask for more detail
Which details has the person already given?	Ask only for the missing ones
After the steps: fixed, not fixed, stuck, or wants a person?	Close the ticket, try the next fix, or hand over

Why this matters: if Jev is unsure, the robot asks a question. If the rules say "this needs a person", the rules win, however sure Jev is. And because every Jev answer and its percentages are saved, an agent can always see *why* the robot did something.

There is also a pretend brain, the **MockBrain**, which guesses from keywords. It's used for automated tests and offline demos, so tests can run without the internet.

6 Where the answers come from

Help articles, questions with buttons, and an approved list of apps.

Help articles

The project's dataset has 31 help articles, but they are written for IT staff ("Attempt 1: clear client cache, restart client"). So each one also has an **employee version**, written in plain words with real steps: where to click in Windows 11, what success looks like, how long it takes, and what to have ready. A supervisor approves each version before the robot may show it. If a supervisor edits an article, our updates never overwrite their edit.

Some articles are the robot's to try; others need a person:

- **Two tries, then a person:** VPN, Outlook, printers, meetings, Wi-Fi and most others.
- **Safe checks first, then the hardware team:** a laptop that won't turn on gets "try another charger, hold the power button for 20 seconds" before anyone is called.
- **The robot does it after an identity check:** password reset, account unlock, moving your authenticator to a new phone.
- **Always a person:** security incidents, admin rights, brand-new software, a lost phone.

Questions with buttons

Each category needs certain details (which app, which printer, which phone). The file `content/field_questions.json` says how to ask each one kindly and which answer buttons to show, so people tap instead of type. Details the robot already knows are taken from the person's profile.

The approved catalogue

When someone asks for software or access, the robot checks `content/catalog.json`, a list of:

- **Business systems** people get access to (Jira, Salesforce): it shows how to request access.
- **Standard software** anyone can install (Zoom, Slack): install it from the Company Portal.
- **Licensed software** (Tableau, Photoshop): Company Portal, or a licence request.
- **Anything not listed** (for example "ollama"): it needs a security and licensing review first.

7 The safety rules

The things the robot must never do, and how the code makes sure.

Rule	How it's kept
Never invent a fix	There is no sentence-writing AI. Every reply is an approved article or a fixed message.
Unsure means ask, or a person	Every Jev answer has a confidence score, and the code checks it.
Never miss danger	The safety check runs on every single message. Hacking or safety goes straight to a person as P1 .
Only change your own account	"Reset my colleague's password" is refused politely.
Prove it's you before any account change	Identity levels V0 to V4. A reset needs an approval on your registered authenticator app. If you don't have one, a person verifies you by phone; the robot never pretends.
No admin powers by robot	Admin rights and wiping a device always need approval from people.
Keep passwords out	Passwords typed in chat are hidden; passwords in screenshots are blurred.
Everything is recorded	An append-only log: entries can be added but never changed or deleted, enforced by the database itself.
Respect privacy	Written with India's DPDP Act in mind: screenshots deleted after 90 days, personal text removed 24 months after a ticket closes, "download my data", and a log of every time staff open a ticket.

8 Tricky situations it handles

Things we found while testing with real conversations, and what the robot does now.

Situation	What the robot does
Two different problems in one chat	Asks "Shall I open a separate ticket?" Yes starts a new conversation for it; No adds it to the first ticket. One problem, one ticket, one conversation.
"Thank you"	Says "You're welcome!", closes the conversation, and starts a fresh one for the next message.
Locked out, authenticator doesn't work	Explains it can't confirm who you are in chat, and that Identity Security will call you to verify you.
Unlocked, but still can't sign in	Explains the password is probably the problem, then resets it.
New phone	Asks if you still have the old phone (that's where the approval goes) before doing anything.
Possible phishing email	Asks a few questions first. If you clicked a link or typed a password, the same ticket becomes P1 at once.
Lost phone	Says what to do right now (Find My, don't erase it yourself, police report) and handles it around the clock.
"People can't hear me" (is it many people or just you?)	If unsure, asks "just you, or colleagues too?" Many people means an outage, so no laptop troubleshooting.
Something unclear, like "os crash"	Asks what happens, when, and any error; then how badly it stops your work, which sets the priority.
Not an IT question ("canteen menu")	Says it doesn't look like IT, and still offers a button to open a ticket in case it's wrong.
Frustrated or "status update?"	Answers about <i>this</i> ticket, says sorry if it's slow, calls an overdue reply overdue, and offers "Ask the team to prioritise this", which notifies the team lead once.
A screenshot of a different problem	Asks whether it's a separate problem instead of acting on it.

Situation	What the robot does
A message in another language	Says it can only read English for now, and offers a person.

9 Time promises

How fast someone will reply, and how fast it should be fixed.

A promise about time is called an **SLA** (service level agreement). Each ticket has two clocks: **first reply** (when a person will answer) and **fix** (when it should be solved). The clock only counts working hours, 9:00 am to 6:00 pm Monday to Friday, India time, skipping holidays. Some problems are too important to wait for Monday, so their clock runs around the clock: every **P1**, sign-in, MFA, security problems and a lost phone.

The dataset gave every category the same promise, which meant a password reset had the same 3-day fix target as a laptop repair. So `content/service_targets.json` sets realistic times per category. Some examples at **P3**:

Kind of problem	First reply	Fixed within
Password, unlock, MFA	30 minutes	4 hours
VPN, email, meetings, network	1 hour	1 working day
Printer	2 hours	1 working day
Hardware, software, access	4 hours	2 working days

The robot always tells people the exact time and date, like "by 1:00 pm tomorrow (Mon 5 Oct)". While the team waits on the employee, the fix clock pauses.

10 The screens

What each person sees, and what each page is for.

Who	Page	What it's for
Everyone	Sign in, My account	Employee ID and password; change your password
Employee	Chat	Talk to the robot, attach screenshots, rate answers 👍 / 👎

Who	Page	What it's for
Employee	Ticket form	Fill a form instead of chatting; the robot checks it and offers the top fix first
Employee	My tickets	Status, reply times, the conversation, reply or reopen
Agent	My queue	The team's tickets, with filters (mine, unowned, waiting) and timers
Agent	Ticket	All fields, the robot's reasoning, screenshots, comments to the employee, private work notes, checklist, Resolve, Reassign
Agent	Bot answers log	Every answer the robot gave and whether it worked
Agent	Knowledge	Help articles, questions no article answered (knowledge gaps), saved replies
Agent	Charts, Agent performance, Shift	How the queue is doing; who handled what; a shift summary
Supervisor	All of the above, for every team	Plus approving articles
Supervisor	Corrections log	Every time an agent fixed the robot's category or priority, and why
Supervisor	User accounts	Create users, reset passwords, change roles, disable accounts

11 Map of the code

Every folder and important file, in one sentence each.

File or folder	What it does
<code>api.py</code>	The office manager: every web address the screens can call, sign-in checks, and permissions
<code>app.py</code>	The front door of the screens: sign-in page, then the right menu for each kind of person
<code>views/</code>	One file per page: chat, form, my tickets, queue, ticket, knowledge, charts, performance, shift, corrections, users, account
<code>ui/</code>	Shared screen pieces: talking to the API, styles, alerts and the P1 dialog
<code>servicedesk/orchestrator.py</code>	The rule-follower: the whole conversation, from first message to hand-off
<code>servicedesk/brain.py</code>	Every question we ask Jev, plus the offline MockBrain
<code>servicedesk/knowledge.py</code>	Reads the datasets: categories, routing, priority matrix, time targets, help articles
<code>servicedesk/business_hours.py</code>	The clock: working hours, holidays, around-the-clock rules, and "by 3:00 pm today (Sat 3 Oct)" wording
<code>servicedesk/store.py</code>	The notebook: SQLite or Postgres, table set-up, the append-only log
<code>servicedesk/tools.py</code>	Pretend IT tools (reset, unlock, MFA reset, asset lookup), ready to be swapped for real ones
<code>servicedesk/config.py</code>	All settings in one place, read from the <code>.env</code> file
<code>servicedesk/services/</code>	Helpers: tickets, sign-in, alerts, notifications, charts, help articles, screenshots, privacy
<code>servicedesk/content/</code>	Words and lists you can change without code: employee articles, questions, time targets, the catalogue
<code>datasets/</code>	The 28 rulebook files from the project specification

File or folder	What it does
<code>worker.py</code>	The background helper: re-alerts, emails, auto-close, clean-up
<code>healthwatch.py</code>	The watchdog that checks the service desk is alive
<code>manage.py</code>	The admin command-line tool
<code>evaluate.py</code>	Runs many test conversations against the real Jev and scores them
<code>tests/</code>	100 automatic checks (see the next chapter)
<code>docs/</code>	Guides: run guide, deploy, go-live checklist, privacy, pilot plan, test plan, diagrams
<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>Caddyfile</code>	Packing everything into containers for a real server, with HTTPS
<code>.github/workflows/</code>	GitHub's robots: one runs the tests, one builds and publishes the app

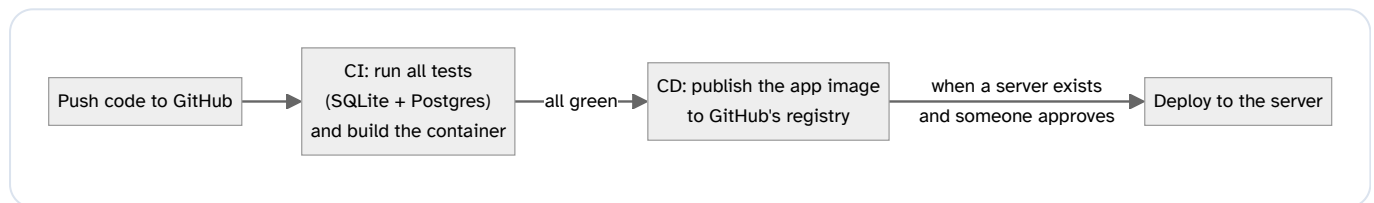
12 Testing and delivery

How we know it works, and how it gets to a server.

Three kinds of testing

- **Automatic checks:** 100 tests that pretend to be employees, agents and supervisors and check every answer. They use the MockBrain, so they run offline in about two minutes, on both SQLite and Postgres.
- **Live conversations:** after every change, real conversations are run against the real Jev, and every reply is read by a person.
- **By hand:** `docs/TEST_PLAN.md` lists, for every category, an easy, a medium and a hard message to type and what should happen, plus checklists for the agent and supervisor screens.

Delivery (CI/CD)



When code is pushed to GitHub, the tests run automatically. If they all pass, the app is packed into a container image and published. Deploying to a server waits until the company has a server, and always needs a person's approval. Going back to an older version means running the deploy again with that version's number.

13 How to run it

Two windows, one browser.

- 1 **Get the code** from GitHub (`Pavithra-SSG/SD-Assistant`) and set up Python's virtual environment with the listed packages.
- 2 **Create the `.env` file** with the secret settings (the Jev key, the demo password). It's never put on GitHub.
- 3 **Window 1, the office manager:** `.\.venv\Scripts\python.exe -m uvicorn api:app --port 8000`
- 4 **Window 2, the screens:** `.\.venv\Scripts\python.exe -m streamlit run app.py`
- 5 **Open** `http://localhost:8501` and sign in: employees EMP1001 to EMP1050, agents EMP2002 to EMP2015, supervisor EMP2001.

After changing code, restart the window you changed (usually the API). On a real server, Docker runs everything, including the worker and the watchdog; [docs/DEPLOY.md](#) explains how.

14 What we built, day by day

The project so far, in order.

Phase 1-2 · late Sep 2026

The core: chat, ticket form, 12 categories, two-attempt fixes, hand-offs, the API as the only writer, sign-in for every role, the 14 scenarios from the specification as tests.

Production round · 30 Sep

Postgres, the background worker, the admin tool, Docker, HTTPS and backups.

Phase 4 · 30 Sep

Screenshots read on our own server, approved employee articles, feedback, business-hours clocks, privacy and retention, emails and Teams messages, the watchdog.

2 Oct

Online demo, the code on GitHub, automatic tests and publishing (CI/CD), the README with diagrams.

3 Oct

Fixes from hands-on testing: kinder wording, answer buttons, exact dates, security questions before hand-off, frustrated follow-ups, catch-all "Something else", clean new problems in the same chat.

3-4 Oct

Realistic answers: rewritten help articles, identity checks that match what people say, realistic time targets, answers before any hand-off, one ticket per conversation, the approved catalogue.

4 Oct

Staff screens checked page by page: the P1 alert no longer stacks up, Take over works, the checklist no longer un-ticks itself. The hands-on test plan.

15 Word list

The grown-up words in this page, explained.

Ticket	A numbered note about one problem, like <code>TKT-3001</code> , so nothing is forgotten.
Queue	A team's line of tickets waiting for help.
Category	The kind of problem (one of 12), which decides the team.
Priority (P1–P4)	How urgent a ticket is. <code>P1</code> is an emergency, <code>P4</code> can wait.
Triage	Sorting a problem: what kind, how many people, how urgent. Hospitals use the same word.
Escalation, hand-off	Passing a ticket from the robot to a person.
SLA	A promise about time: when someone will reply and when it should be fixed.
Knowledge base (KB)	The collection of help articles.
API	The office manager: the set of web addresses the screens use to ask for things.
Database	The big notebook where everything is stored.
OCR	Reading the text inside a picture.
VPN	A secure tunnel from home into the company network.
MFA, authenticator	A second proof it's you, usually an approval on a phone app, on top of your password.
Company Portal	The company's app store for approved software.
Outage, incident	Something broken for many people at once, handled as one shared ticket.
Agent, supervisor	An IT helper in a team; the person who leads all the teams.
Token	A pass your browser gets after signing in, so you don't type your password on every page.
Append-only log	A record you can add to but never change or delete.
CI/CD	GitHub robots that test the code and package it automatically whenever it changes.
Jev	The AI model that answers only multiple-choice questions, with a confidence score.

SD-Assistant · github.com/Pavithra-SSG/SD-Assistant · Written 4 October 2026, describing the project up to that date.